Uma **API REST** é uma interface de programação de aplicações que segue os princípios do **REST** (*Representational State Transfer* ou Transferência de Estado Representacional), um estilo arquitetural criado em 2000 por Roy Fielding. [1, 2, 3]

Ela serve como uma ponte de comunicação entre diferentes sistemas (como um aplicativo móvel e um servidor) utilizando a internet. [1, 2]

Como funciona

A comunicação em uma API REST funciona no modelo **cliente-servidor** e utiliza o **protocolo HTTP** (o mesmo usado para navegar em sites). O cliente faz um pedido (*requisição*) e o servidor processa e devolve uma resposta (*response*). [1, 2, 3, 4, 5, 6]

Para realizar as operações, a API REST utiliza os **métodos HTTP** (verbos) aplicados a uma URL específica (*endpoint*):

- **GET:** Solicita a leitura ou recuperação de dados.
- **POST:** Envia dados para criar um novo recurso.
- **PUT / PATCH:** Atualiza um recurso existente (o `PUT` substitui o objeto inteiro; o `PATCH` atualiza apenas campos específicos).
- **DELETE:** Remove um recurso. [1, 2]

Os dados trocados entre o cliente e o servidor são comumente estruturados no formato **JSON** (*JavaScript Object Notation*), embora XML ou outros formatos também possam ser utilizados. [1, 2, 3]

Princípios principais da arquitetura REST

Para que uma API seja considerada RESTful (aderente ao REST), ela costuma seguir estas regras fundamentais:

- **Cliente-Servidor:** Há separação clara entre quem consome a interface (cliente) e quem processa os dados e a lógica (servidor). [1, 2]
- **Sem estado (*Stateless*):** Cada requisição do cliente para o servidor deve conter todas as informações necessárias para ser compreendida; o servidor não guarda o histórico de sessões anteriores do cliente. [1, 2, 3]
- **Cacheável:** As respostas do servidor devem indicar se podem ou não ser armazenadas em cache para evitar requisições repetidas e melhorar o desempenho. [1, 2, 3]
- **Interface uniforme:** Padroniza as URLs (recursos) e os métodos HTTP de forma consistente e autodescritiva. [1, 2, 3]

- **Sistema em camadas:** O cliente pode estar se comunicando com intermediários (como balanceadores de carga, firewalls ou proxies) sem precisar saber se está no servidor final. [1, 2, 3]

Vantagens

- **Leveza e Desempenho:** Por usar JSON e o protocolo HTTP de forma direta, é mais rápida e consome menos banda que modelos mais antigos como o SOAP. [1, 2, 3]
- **Independência de Tecnologia:** O servidor e o cliente podem ser desenvolvidos em linguagens e plataformas totalmente diferentes (ex: backend em Node.js ou Python conversando com frontend em React ou app iOS). [1, 2]
- **Escalabilidade:** O modelo *stateless* e a facilidade de uso em microsserviços permitem que sistemas cresçam com mais facilidade. [1, 2, 3]

Saiba mais sobre a definição no artigo da IBM sobre REST APIs ou veja os conceitos explicados pela Red Hat sobre API REST. [1, 2]